

HTTP Signature Keys

draft-hardt-httpbis-signature-key-07

Dick Hardt (Hellō) & Thibault Meunier (Cloudflare)

httpbis — IETF 126 Vienna — July 2026

Agenda

1. Quick overview: what is Signature-Key?
2. What's happened since IETF 125
3. New mechanisms: `sigkey` parameter, Signature-Error
4. New schemes: `jkt-jwt`, `self-jwt`
5. Implementation, adoption & demos
6. **The bigger picture: client authentication for HTTP**

Quick Overview: The Missing Piece

- HTTP Message Signatures (RFC 9421) — proof of possession at the application layer
 - works through proxies and CDNs, unlike mTLS
- To verify a signature, the verifier needs the signer's **public key**
- RFC 9421 intentionally leaves **key distribution** to application protocols
- **Signature-Key** fills that gap: a header that carries the key — or where to find it — inline with the signed message

```
Signature-Input: sig=("@method" "@authority" "@path" "signature-key"); created=1752710000
Signature:      sig=:MEQCIA5...
Signature-Key:  sig=hwk;kty="0KP";crv="Ed25519";x="JrQLj..."
```

Quick Overview: Schemes for Different Trust Models

Scheme	Model	Identity
hwk	Pseudonymous	Key thumbprint
jkt-jwt	Pseudonymous + delegation	Stable enclave key thumbprint
jwks_uri	Identified	HTTPS URL + well-known metadata
jwt	Delegated	JWT issuer signs for instances
self-jwt	Identified + claims	Issuer is the signer
x509	PKI	Certificate subject

The scheme tells the verifier the **trust model** and **verification procedure** — extensible via IANA registry

Since IETF 125: Five Revisions

Draft	Date	Highlights
-03	Apr 4	jkt-jwt scheme, dwk parameter, early validation
-04	Apr 9	Renamed " HTTP Signature Keys "; sigkey parameter; Signature-Error header
-05	Jun 17	Implementer feedback — SSRF defenses, caching rules
-06	Jul 2	self-jwt scheme
-07	Jul 4	Editorial; AAuth & Email Verification cited as users

Also: presented client identity framing at the **web bot auth interim** (April 13)

Now Three Mechanisms, Not One

The spec grew from a single header to a set of building blocks:

1. **Signature-Key** (request) — distributes the verification key, six schemes
2. **sigkey** parameter for **Accept-Signature** (RFC 9421 §5) — server signals the *type* of key it requires: **jkt** (pseudonymous), **uri** (identified), **x509** (PKI)
3. **Signature-Error** (response) — structured verification errors (**invalid_signature** , **unknown_key** , **expired_jwt** , ...) with an IANA error code registry

Incremental Adoption — Zero Coordination

Servers can adopt signatures **without breaking existing clients**:

1. Server responds 429 / 401 / 402 with `Accept-Signature: sig=();sigkey=jkt`
2. Client retries with a signed request and Signature-Key
3. Verification failures return **Signature-Error** so clients can self-diagnose
 - No pre-registration, no out-of-band setup
 - Unsigned traffic keeps working — signed traffic gets better treatment
 - Rate limit by key thumbprint instead of by IP

New Scheme: jkt-jwt (from preview → shipped)

Previewed at IETF 125, now fully specified in -03:

- Hardware enclave key (slow, secure) delegates to an **ephemeral software key** (fast) via a self-issued JWT
- Stable identity: `urn:jkt:sha-256:<thumbprint>` (JWK Thumbprint URI)
- TOFU trust model (RFC 7435) — hardware-rooted identity, software signing speed

New Scheme: self-jwt (-06)

- **JWT issuer and HTTP signer are the same party** — no `cnf` claim
- One key, discovered via `{iss}/.well-known/{dwk}`, verifies both JWT and HTTP signature
- Carries claims (`sub`, `aud`, ...) alongside an identified signature

Hardened by Implementation

Implementer feedback from Joshua Gay (**sidecat**) drove -05:

- **Egress admission checklist** for `jwks_uri` fetches:
HTTPS only, size/timeout limits, redirect policy, reject private/loopback addresses, DNS rebinding defense
- **Caching rules**: once-per-minute fetch floor; one refresh-and-retry on same-`kid` signature failure before returning `unknown_key`

The spec is being shaped by people building it, not just reviewing it

Adoption: Specs Building on Signature-Key

- **AAuth** (draft-hardt-oauth-aauth-protocol) — all parties use Signature-Key to distribute the keys that verify their signed requests
- **Email Verification** (draft-hardt-email-verification, DISPATCH this week) — uses `hwk` to convey the browser's public key so the issuer can bind it into the verification token
- **web bot auth** — architecture uses the same primitives: RFC 9421 signatures + key discovery; Signature-Key discussed at the April interim

Designed as **general-purpose building blocks** — protocols adopt without coordination

Demos: Replacing API Keys with Signature Keys

Lightning demos at **AAuth Night** (San Francisco, July 1, 2026):

Demo	Who
Vestauth — auth for agents, from the creator of <code>dotenv</code> / <code>dotenvx</code>	Scott Motte
Keycard — identity and access platform for AI agents	Jared Hanson
LoginID — strong authentication for agents acting for users	Jesse Ariss
MailChannels — email sending for AI agents	Ken Simpson
AAuth Web Agent — live AAuth protocol demo	Dick Hardt

Secret managers are moving from **distributing shared secrets** to **distributing signature keys** — proof of possession instead of API keys

The Bigger Picture: Client Identity Is Unsolved

Server identity — solved

- DNS + TLS + Let's Encrypt: globally unique names, cryptographic binding, no gatekeeper

Client identity — not solved

- mTLS never escaped the enterprise perimeter
- On the open web, every client falls back to **shared secrets**: API keys, passwords

API keys are bearer tokens — whoever has the key *is* the client

- 39M secrets leaked on GitHub in 2024; 90% still valid 5 days later

The Same Primitives Solve It Everywhere

Client	Solved today with
Bot / crawler	web bot auth (in progress)
AI agent calling an API	API keys
IoT device calling home	Proprietary attestation
Mobile app instance	Platform-specific attestation
CI/CD pipeline	GitHub OIDC, SPIFFE
Server-to-server	mTLS or bearer tokens

Every answer is **platform-specific, complex, or a shared secret** —
yet all of them reduce to: *sign the request, let the verifier find the key*

Call to Action

We need to solve how we authenticate clients for HTTP

- HTTP Message Signatures gave us the signature
- Signature-Key provides the missing key distribution — spanning pseudonymous → identified → delegated → PKI trust models
- Proof of possession, not shared secrets

Asks

- Review the draft — especially `sigkey` negotiation and Signature-Error
- Implementers: we want more feedback like sidecat's
- Is there interest in adopting this work in httpbis?
— client authentication is bigger than bots

Questions / Discussion

- dick.hardt@gmail.com
- draft-hardt-httpbis-signature-key-07
- <https://github.com/dickhardt/signature-key>